

Созданные индексы

Атрибут	Операция	Тип индекса	Команда создания
Enrollment.progress	Поиск по диапазону (BETWEEN, >, <)	B-tree	CREATE INDEX idx_enrollment_progress ON "Enrollment"(progress);
User.role	Фильтрация (WHERE role = ...)	B-tree	CREATE INDEX idx_user_role ON "User"(role);
User.name	Сортировка (ORDER BY name)	B-tree	CREATE INDEX idx_user_name ON "User"(name);
Course.title	Поиск по подстроке (LIKE '%...%')	GIN (pg_trgm)	CREATE EXTENSION pg_trgm; CREATE INDEX idx_course_title_trgm ON "Course"USING gin (title gin_trgm_ops);
Enrollment.course_id	Группировка (GROUP BY course_id)	B-tree	CREATE INDEX idx_enrollment_course_id ON "Enrollment"(course_id);
Enrollment.student_id	JOIN (внешний ключ)	B-tree	CREATE INDEX idx_enrollment_student ON "Enrollment"(student_id);
Enrollment.enrolled_at	Сортировка по дате	B-tree	CREATE INDEX idx_enrollment_enrolled_at ON "Enrollment"(enrolled_at);

Сравнение производительности до и после создания индексов

Индекс 1: поиск по диапазону (progress)

```

1 -- Execution Time: 492 ms
2 EXPLAIN (ANALYZE, BUFFERS)
3 SELECT * FROM "Enrollment"
4 WHERE progress BETWEEN 80 AND 100
5 ORDER BY enrolled_at;

```

План: Seq Scan + Sort

```

1 -- Execution Time: 464 ms
2 CREATE INDEX idx_enrollment_progress ON "Enrollment" (progress);
3 EXPLAIN (ANALYZE, BUFFERS)
4 SELECT * FROM "Enrollment"
5 WHERE progress BETWEEN 80 AND 100
6 ORDER BY enrolled_at;

```

План: Bitmap Index Scan + Bitmap Heap Scan + Sort

Индекс 2: фильтрация и сортировка (role, name)

```
1 -- Execution Time: 482 ms
2 EXPLAIN ANALYZE
3 SELECT id, name, email FROM "User"
4 WHERE role = 'student'
5 ORDER BY name;
```

План: Seq Scan + Sort

```
1 -- Execution Time: 469 ms
2 CREATE INDEX idx_user_role ON "User" (role);
3 CREATE INDEX idx_user_name ON "User" (name);
4 EXPLAIN ANALYZE
5 SELECT id, name, email FROM "User"
6 WHERE role = 'student'
7 ORDER BY name;
```

План: Index Scan (idx_user_name) + Filter

Индекс 3: поиск по подстроке (LIKE)

```
1 -- Execution Time: 506 ms
2 EXPLAIN ANALYZE
3 SELECT * FROM "Course"
4 WHERE title LIKE '%Python%';
```

План: Seq Scan

```
1 -- Execution Time: 469 ms
2 CREATE EXTENSION IF NOT EXISTS pg_trgm;
3 CREATE INDEX idx_course_title_trgm ON "Course" USING gin (title
  gin_trgm_ops);
4 EXPLAIN ANALYZE
5 SELECT * FROM "Course"
6 WHERE title LIKE '%Python%';
```

План: Bitmap Index Scan (GIN) + Bitmap Heap Scan

Запрос	До индекса	После индекса	Ускорение
Поиск по диапазону (progress)	492 ms	464 ms	5.7%
Фильтрация + сортировка (role, name)	482 ms	469 ms	2.7%
Поиск по подстроке (LIKE)	506 ms	469 ms	7.3%

Анализ производительности сложных запросов

Для оценки влияния индексов на выполнение запросов с соединениями, агрегацией и сортировкой были подготовлены четыре запроса средней и высокой сложности. Замеры времени выполнения произведены до и после создания индексов.

Запрос 1: количество студентов, записанных на каждый курс

```
1 -- Before indexes (Execution Time: 491 ms)
2 EXPLAIN (ANALYZE, BUFFERS)
3 SELECT c.title AS course_name, COUNT(e.id) AS students_count
4 FROM "Course" c
5 LEFT JOIN "Enrollment" e ON c.id = e.course_id
6 GROUP BY c.id, c.title
7 ORDER BY students_count DESC;
```

План: Seq Scan + Hash Join + HashAggregate + Sort

```
1 -- After indexes (Execution Time: 479 ms)
2 -- Indexes used: idx_enrollment_course_id
3 EXPLAIN (ANALYZE, BUFFERS)
4 SELECT c.title AS course_name, COUNT(e.id) AS students_count
5 FROM "Course" c
6 LEFT JOIN "Enrollment" e ON c.id = e.course_id
7 GROUP BY c.id, c.title
8 ORDER BY students_count DESC;
```

План: Index Scan (idx_enrollment_course_id) + Nested Loop + HashAggregate + Sort

Запрос 2: средний прогресс по курсам (с фильтром HAVING)

```
1 -- Before indexes (Execution Time: 496 ms)
2 EXPLAIN (ANALYZE, BUFFERS)
3 SELECT c.title AS course_name, AVG(e.progress) AS avg_progress
4 FROM "Course" c
5 JOIN "Enrollment" e ON c.id = e.course_id
6 GROUP BY c.id, c.title
7 HAVING AVG(e.progress) > 50
8 ORDER BY avg_progress DESC;
```

План: Seq Scan + Hash Join + HashAggregate + Filter + Sort

```
1 -- After indexes (Execution Time: 486 ms)
2 -- Indexes used: idx_enrollment_course_id, idx_enrollment_progress
3 EXPLAIN (ANALYZE, BUFFERS)
4 SELECT c.title AS course_name, AVG(e.progress) AS avg_progress
5 FROM "Course" c
6 JOIN "Enrollment" e ON c.id = e.course_id
7 GROUP BY c.id, c.title
8 HAVING AVG(e.progress) > 50
9 ORDER BY avg_progress DESC;
```

План: Index Scan (idx_enrollment_course_id) + Nested Loop + HashAggregate + Filter + Sort

Запрос 3: Топ-10 студентов по суммарному прогрессу

```
1 -- Before indexes (Execution Time: 512 ms)
2 EXPLAIN (ANALYZE, BUFFERS)
3 SELECT u.name AS student_name, SUM(e.progress) AS total_progress, COUNT
   (e.course_id) AS courses_count
4 FROM "User" u
5 JOIN "Enrollment" e ON u.id = e.student_id
6 WHERE u.role = 'student'
7 GROUP BY u.id, u.name
8 ORDER BY total_progress DESC
9 LIMIT 10;
```

План: Seq Scan + Hash Join + HashAggregate + Sort + Limit

```
1 -- After indexes (Execution Time: 480 ms)
2 -- Indexes used: idx_enrollment_student_id, idx_user_role,
   idx_user_name
3 EXPLAIN (ANALYZE, BUFFERS)
4 SELECT u.name AS student_name, SUM(e.progress) AS total_progress, COUNT
   (e.course_id) AS courses_count
5 FROM "User" u
6 JOIN "Enrollment" e ON u.id = e.student_id
7 WHERE u.role = 'student'
8 GROUP BY u.id, u.name
9 ORDER BY total_progress DESC
10 LIMIT 10;
```

План: Index Scan (idx_user_role, idx_user_name) + Nested Loop + HashAggregate + Sort + Limit

Запрос	До индексов (мс)	После индексов (мс)	Ускорение
Количество студентов по курсам	491	479	2,4%
Средний прогресс (HAVING)	496	486	2,0%
Топ-10 студентов по прогрессу	512	480	6,3%

Вывод: созданные индексы (на course_id, student_id, progress, role) позволили сократить время выполнения запросов в среднем на 2–6%. Наибольший эффект достигнут для запроса с фильтрацией, сортировкой и ограничением (топ-10 студентов). Небольшое ускорение объясняется малым объёмом данных в таблицах Course, Module, Lesson (менее 20 записей) и ограничением уникальности пар в Enrollment (не более 3000 записей).

```
1 -- Indexes to speed up JOINS and GROUP BY on Enrollment
2 CREATE INDEX idx_enrollment_course_id ON "Enrollment" (course_id);
3 CREATE INDEX idx_enrollment_student_id ON "Enrollment" (student_id);
4 CREATE INDEX idx_enrollment_progress ON "Enrollment" (progress);
5 CREATE INDEX idx_user_role ON "User" (role);
6 CREATE INDEX idx_user_name ON "User" (name);
```

Аномалия: Non-repeatable Read

Non-repeatable read возникает, когда в рамках одной транзакции повторный запрос возвращает разные результаты из-за изменений, сделанных другой транзакцией.

Шаг	Транзакция T1	Транзакция T2
1	BEGIN;	
2	SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	
3	SELECT balance FROM accounts WHERE id = 1;	→ Результат: 1000
4		BEGIN;
5		UPDATE accounts SET balance = 800 WHERE id = 1;
6		COMMIT;
7	SELECT balance FROM accounts WHERE id = 1;	→ Результат: 800 (аномалия)
8	COMMIT;	

```
1 -- Terminal 1
2 BEGIN;
3 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
4 SELECT balance FROM accounts WHERE id = 1; -- 1000
5 -- Terminal 2
6 BEGIN;
7 UPDATE accounts SET balance = 800 WHERE id = 1;
8 COMMIT;
9 -- Terminal 1
10 SELECT balance FROM accounts WHERE id = 1; -- 800
11 COMMIT;
```

При уровне изоляции READ COMMITTED транзакция T1 видит все изменения, зафиксированные другими транзакциями. После того как T2 закоммитила своё изменение, T1 при повторном чтении видит новое значение баланса. Для устранения non-repeatable read необходимо повысить уровень изоляции до REPEATABLE READ:

```
1 -- Terminal 1
2 BEGIN;
3 SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
4 SELECT balance FROM accounts WHERE id = 1; -- 1000
5 -- Terminal 2
6 BEGIN;
7 UPDATE accounts SET balance = 800 WHERE id = 1;
8 COMMIT;
9 -- Terminal 1
10 SELECT balance FROM accounts WHERE id = 1; -- 1000
11 COMMIT;
```

Уровень изоляции REPEATABLE READ гарантирует, что транзакция видит снимок данных (snapshot) на момент своего начала. Все последующие чтения в рамках этой транзакции будут видеть те же самые данные, даже если другие транзакции их изменили и зафиксировали.

Аномалия: Phantom Read

Phantom read возникает, когда при повторном выполнении запроса с условием в результатах появляются новые строки, которые были вставлены другой транзакцией.

Шаг	Транзакция T1	Транзакция T2
1	BEGIN;	
2	SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	
3	SELECT COUNT(*) FROM "Enrollment" WHERE course_id = 2;	→ Результат: 1000
4		BEGIN;
5		INSERT INTO "Enrollment"(student_id, course_id, progress) VALUES (1001, 2, 50);
6		COMMIT;
7	SELECT COUNT(*) FROM "Enrollment" WHERE course_id = 2;	→ Результат: 1001 (аномалия)
8	COMMIT;	

```
1 -- Terminal 1
2 BEGIN;
3 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
4 SELECT COUNT(*) FROM "Enrollment" WHERE course_id = 2; -- 1000
5 -- Terminal 2
6 BEGIN;
7 INSERT INTO "Enrollment" (student_id, course_id, progress) VALUES
  (1001, 2, 50);
8 COMMIT;
9 -- Terminal 1
10 SELECT COUNT(*) FROM "Enrollment" WHERE course_id = 2; -- 1001
11 COMMIT;
```

При уровне изоляции READ COMMITTED каждая новая строка, вставленная и закоммиченная другой транзакцией, становится видимой для текущей транзакции при повторном выполнении запроса. Для устранения phantom read необходимо повысить уровень изоляции до REPEATABLE READ:

```
1 -- Terminal 1
2 BEGIN;
3 SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
4 SELECT COUNT(*) FROM "Enrollment" WHERE course_id = 2; -- 1000
5 -- Terminal 2
6 BEGIN;
7 INSERT INTO "Enrollment" (student_id, course_id, progress) VALUES
  (1002, 2, 60);
8 COMMIT;
9 -- Terminal 1
10 SELECT COUNT(*) FROM "Enrollment" WHERE course_id = 2; -- 1000
11 COMMIT;
```

Уровень REPEATABLE READ защищает от фантомного чтения, так как транзакция работает со снимком данных на момент своего начала. Новые строки, вставленные другими транзакциями, не попадают в этот снимок.

Аномалия: Dirty Read

Dirty read возникает, когда транзакция читает данные, изменённые другой незафиксированной транзакцией. В PostgreSQL эта аномалия невозможна благодаря MVCC.

Шаг	Транзакция T1	Транзакция T2
1	BEGIN;	
2	UPDATE accounts SET balance = 2000 WHERE id = 1;	
3		SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
4		SELECT balance FROM accounts WHERE id = 1;

```
1 -- Terminal 1
2 BEGIN;
3 UPDATE accounts SET balance = 2000 WHERE id = 1; -- no COMMIT
4
5 -- Terminal 2
6 SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
7 SELECT balance FROM accounts WHERE id = 1; -- 1000
```

Даже при уровне READ UNCOMMITTED PostgreSQL возвращает старое значение 1000. Dirty read отсутствует, специального устранения не требуется.